

ϵ -NFA and Regex Equivalence In Lean

Eyal Barnea, Harry Cohen, Noam Jacobi

Winter 25-26 - Spring 26

Abstract

This project (<https://github.com/eyal3741/Formal-Matka/>) proves the equivalence of the computational models ϵ -NFA and regular expressions, as seen in Introduction To Set Theory And Automata (02340129), using the Lean proof assistant. During this project we have uncovered a gap between the definitions of the different types of Finite Automata and updated the definition of ϵ -NFA to be on par with the definitions of both DFA and NFA.

1 Overview Of Subject And Proof Idea

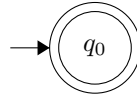
As "We all remember the material of Introduction To Set Theory And Automata, since it is engraved on the tablet of our hearts"¹, it is known that the computational models of Deterministic Finite Automata (DFA), Non-Deterministic Finite Automata (NFA) and Non-Deterministic Finite Automata with ϵ transitions (ϵ -NFA) are proven to have the same expressive power. In order to show their equivalence to Regular Expressions (Regex), we would need to show that for every ϵ -NFA A , there exists a Regex r such that $L(A) = L(r)$, and vice versa. These two proofs (written in short below) turn out to be quite challenging in Lean, as Lean has many tools and lemmas for proving mathematical statements, while for both Regex and automata "their underlying mathematical foundation is extremely poor (in that there is no rich theory underneath them, such as calculus, probability theory, group theory, etc.), and yet they give rise to very intricate problems. Since there are not many mathematical foundations to use, all we can use is our brain. Hence – fun!"²

1.1 $\forall r \exists A$:

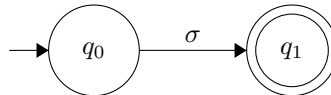
Since Regex is defined using Structural Induction, the proof follows in the same way:

- base:

- The Empty Language: An automaton with no states.
- The Empty Word: An automaton with one state, both starting and accepting, and no transitions.



- A Single Character σ : An automaton with two states, one starting and the other accepting, and a single transition from the starting state to the accepting state using σ .

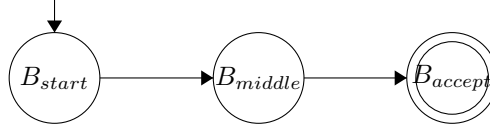
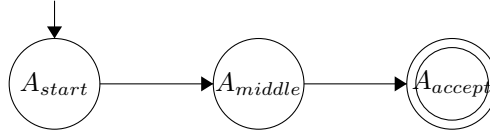


- step:

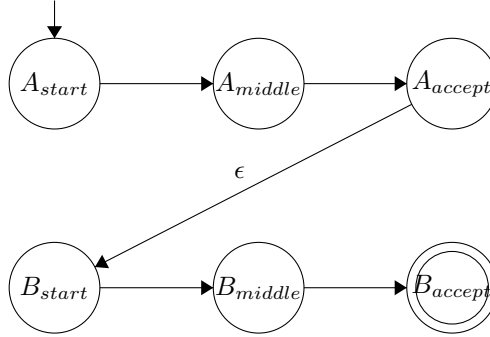
- The + Operator: putting the two automata "side by side"

¹Dr. Sarai Sheinvald, Logic For CS, Lecture 1

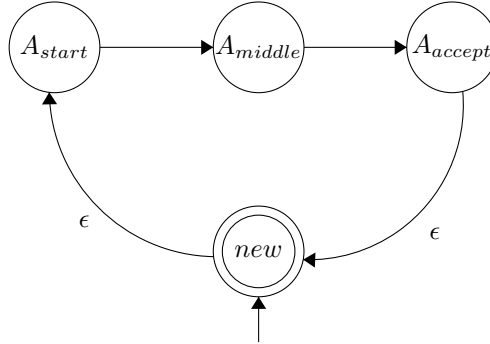
²Assoc. Prof. Shaull Almagor, Automata Logic And Games, Lecture 1



- The $\cdot$ Operator: connecting the first automaton's accepting states with an ϵ transition to the second's starting states



- The $*$ Operator: creating a single state that is both starting and accepting, connecting it to the automaton's starting states, and connecting the automaton's accepting states back to it using ϵ -transitions



1.2 $\forall A \exists r$:

We use Dynamic Programming to build a Regex describing all paths between any two states:

- base: the paths between two states i, j without going through any state in the middle are the transitions between them (or the empty word, if they are the same):

$$r(i, j, 0) = \sum_{\sigma \in \Sigma: \delta(i, \sigma) = j} \sigma + \begin{cases} \epsilon & i = j \text{ or } \delta(i, \epsilon) = j \\ \text{else} & \end{cases}$$

- step: the paths between two states i, j going through only states between $1 \dots k$ are either
 - not going through k , thus going through only $1 \dots k - 1$
 - going through k , in which case they can be divided into three parts: reaching k for the first time, returning to k zero or more times, and finally reaching j .

$$r(i, j, k) = r(i, j, k - 1) + r(i, k, k - 1) \cdot r(k, k, k - 1)^* \cdot r(k, j, k - 1)$$

2 Hardships Encountered

2.1 $\forall r \exists A$:

2.1.1 Working With Automata

In Lean automata, ϵ -NFAs are defined over a universe of states (and for the automata to be well defined, it is the responsibility of the user to make sure they use a finite set. Seeing this, for ease of usage we chose to work over a universe of $\mathbb{N}$), where the reachable states are implicitly defined by the transition function, and any reference to a path within them is only in the form of the proposition "the word x is a path between the states i and j ". In addition, said universe of states is not defined as finite. Thus, the following hardships emerge:

- If two automata share the names of some of their states, there is a need to rename them before using the automata together in any construction. Additionally, since the states of each automaton are implicitly defined and may or may not be finite, the renaming must not take into account the size of the state group.
- Concrete paths do not exist, as no information on any path is available.

In order to deal with said problems, we did the following:

- We used the injective functions $f_0(x) = 2x$ and $f_1(x) = 2x + 1$, where $\text{Img}(f_0) \cup \text{Img}(f_1) = \mathbb{N}$ and $\text{Img}(f_0) \cap \text{Img}(f_1) = \emptyset$, to rename the states back into the same universe without any collisions.
- We construct the automata in such a way that by only using the propositions we can deduce the critical information about the path, skipping over any need for concrete paths.

2.1.2 Automata Equivalence

In order to prove the steps of the structural induction, the following tools needed to be developed:

- Renaming Equivalence: After using one of the renaming functions (f_0, f_1) , the resulting automaton accepts the same words as the original, and has the same paths.
- Automata Containment: Let $A = \langle Q^A, Q_0^A, \Sigma, \delta^A, F^A \rangle$, $B = \langle Q^B, Q_0^B, \Sigma, \delta^B, F^B \rangle$. If $B \subseteq A$ (i.e. $Q^B \subseteq Q^A$, $Q_0^B \subseteq Q_0^A$, $\delta^B \subseteq \delta^A$ as we consider the transition functions as groups of transitions, $F^B \subseteq F^A$), a path in B implies the existence of the same path in A .

2.1.3 Composition

The main hardship involved in the composition was separating the accepted words of the new automaton back into words accepted by the original automata. In the new construction, the states of the two original automata have different parities, and transitions between them go from the first automaton's accepting states to the second automaton's starting states. To use these properties to separate the accepted words, we needed the following sub-proofs:

- In-automata mod 2 consistency except at the crossing point.
- Irreversibility of changing sub-automata from first to second.
- Determining the existence of a crossing point for every accepted word, given that the starting state lies in the first sub-automaton and the accepting state lies in the second.

2.1.4 Kleene Closure

The main hardship involved in Kleene closure was, like in the composition's case, separating the accepted word into accepted sub-words. The only sign of a new sub-word is derived from the singular added "new" state. In order to make use of it we needed to do the following:

- Rename the original automaton's states into even integers and keep the "new" state, now named 1, as the only odd state.
- Carefully follow the combinatorically astounding number of different cases regarding ϵ -transitions and their roles in the first and new automata to find the first accepted sub word.
- Recursively apply the previously-outlined lemma to find the other sub-words.

2.1.5 Additional Notable Lean Hardships

- At first, defining the renaming functions by using the "match" tools seemed like the perfect match, but because disambiguating the function for use in the proof was difficult and unintuitive, we switched to the "if/else" tools.
- When we used induction, many assumptions that were correct only for the base were dragged along for the steps, making the induction harder to prove in the best cases and incorrect in the common ones. In order to bypass this problem, we used every trick in the Lean textbook, including generalizing, renaming propositions and even opening new environments using "have" and new lemmas to keep the assumptions sterile.
- Proving the Kleene closure case was technically challenging, requiring us to handle many edge cases and use a recursively defined lemma for the different subwords. The recursion was justified by a decrease in word length, and the lemma was used to apply the result (lovingly) nicknamed "lemma of absolute evil".

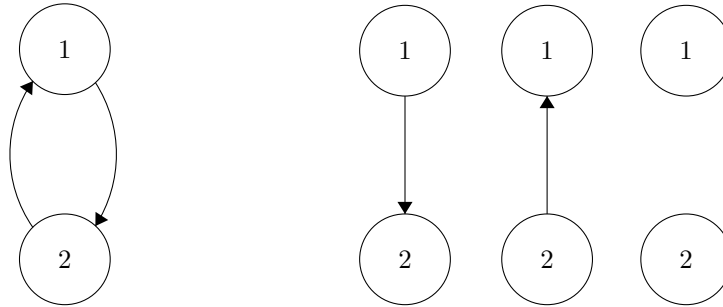
2.2 $\forall A \exists r$:

2.2.1 Finding A Lean Fitting Proof

After reviewing the material of Introduction To Set Theory And Automata, we have found a proof that was quite hand-wavy, in which states of the automaton are being removed one by one, the transitions to and from them merging into Regular Expressions until only two states remain, one starting and one accepting, with one "transition" between them being the equivalent Regex to the entire automaton. This proof requires us to go through the following steps:

- Define "Regex Automata", which read in each transition an unknown number of characters, accepted by the regex of the transition.
- Prove any lemma already proven for other automata that will be needed for the rest of this direction.
- Prove that A_r , created by copying ϵ -NFA A and changing every transition σ to the regex $[\sigma]$, is equivalent to A .
- Prove that every step of state removal and transition merging results in an equivalent regex automaton with a smaller number of states (which are, unless we depart from existing implementations of automata, implicitly defined).

Seeing that, and with the help of Dr. Sarai Sheinvald, we have found the dynamic programming proof (as seen above). This proof, however, was not free of problems: the entire proof relies on our ability to discern whether the path taken goes through the state k , without the ability to talk about concrete paths. As a bypass to the concrete path problem, we have tried to "trim" the automaton, removing any transition that would lead to a state not in $1 - k, j$ but after a great deal of work and effort in this direction we have found a counterexample that shows trimming cannot work - on the following automaton (left), find paths from q_2 to q_2 going only through $1 - 1$:



The only valid paths are $q_2, q_2 - q_1 - q_2$, but no way of trimming the automata (right) can produce only those paths.

Finally, after looking at the definitions of *DFA* and *NFA*, we have discovered that concrete paths are defined in full and due to a mismatch in maintaining the definitions of all automata, ϵ -NFA's definitions were not updated accordingly.

2.2.2 Fixing The Gap In Lean Definition

With the needed changes to Mathlib (<https://github.com/leanprover-community/mathlib4/pull/40329>), many basic tools and lemmas needed to work with paths still needed to be developed:

- Path Support: An ability to talk about the states visited in a given path.
- Path Appending: Concatenation of two paths, one ending where the other begins, including definition, associativity, and support effects.
- Path Containment: Identification of sub-paths, including definition, reflexivity and transitivity.

2.2.3 Additional Notable Lean Hardships

- The Lean regex capabilities proved to be lacking, and as such we needed to define and prove some of the most basic tools - having a set of characters, creating a joint $+$ expression for them all and proving it functions as needed.
- By default, the types of the paths were defined by the transition labels, and so appending paths would cause problems since the different lists were identical in content, yet still not equal. In order to avoid this problem we have used both structures and the equality transport ($\rightrightarrows$) operator.
- Path appending associativity was hard to prove in one of the directions, leading us after a thorough search to the usage of "grind" to solve that specific part.

3 Lessons Learned and Recommendations

- The scope of proofs is hard to determine and seemingly correlates not with the depth of the material, but with the scope and ease of use of the existing tools in the field (and by proxy, with the popularity of the subject among the general Lean user audience).
- Correct division of proofs into lemmas helps both in keeping the proof readable and zeroing in on the hard part - while no proof has been trivial, many hours were saved by identifying the key tools needed.
- Finding the documentation of the relevant mathlib files and identifying key lemmas lead to correct planning and fast proofs.
- The course staff proved to be a powerful resource with vast knowledge, but this resource was underutilized during this project.
- Considering the limitations of Lean while picking a proof significantly changes the "complexity" of the proof - what is easy for the human brain to understand might not be the best for Lean to methodically prove. Making sure to write out every edge case helps to find the hidden assumptions and pinpoint ahead of time the hard parts.